

Optimización conjunta del PLL_SYS y del divisor PIO para síntesis de frecuencias en el RP2040

Proyecto PixiePico Firmware

4 de septiembre de 2026

Resumen

Esta nota presenta un método discreto y exhaustivo para seleccionar conjuntamente `REFDIV`, `FBDIV`, `POSTDIV1`, `POSTDIV2` y el divisor fraccional Q16.8 de una máquina de estados PIO del RP2040. El objetivo es minimizar el error absoluto de una salida cuadrada cuya frecuencia nominal puede variar. Se considera un cristal de 12 MHz, una instrucción PIO para el estado alto y otra para el estado bajo, y el intervalo experimental $125 \text{ MHz} \leq f_{\text{SYS}} \leq 250 \text{ MHz}$. Se presentan resultados para 7,074 MHz, 14,074 MHz y 28,074 MHz.

1. Planteo del problema

El oscilador utiliza PLL_SYS como fuente de reloj de una máquina de estados PIO. El programa PIO ejecuta dos estados por cada período de salida: uno mantiene la señal alta y el siguiente la mantiene baja. Por lo tanto, si D es el divisor de reloj de la PIO,

$$f_{\text{out}} = \frac{f_{\text{SYS}}}{2D}. \quad (1)$$

El divisor PIO se representa en formato Q16.8:

$$D = d_{\text{int}} + \frac{d_{\text{frac}}}{256} = \frac{N}{256}, \quad N = 256d_{\text{int}} + d_{\text{frac}}, \quad (2)$$

donde $0 \leq d_{\text{frac}} \leq 255$. Sustituyendo (??) en (??):

$$f_{\text{out}} = \frac{128f_{\text{SYS}}}{N}. \quad (3)$$

2. Modelo del PLL del RP2040

Para una referencia externa f_{XOSC} , el PLL genera

$$f_{\text{ref}} = \frac{f_{\text{XOSC}}}{\text{REFDIV}}, \quad (4)$$

$$f_{\text{VCO}} = f_{\text{ref}} \text{FBDIV}, \quad (5)$$

$$f_{\text{SYS}} = \frac{f_{\text{VCO}}}{\text{POSTDIV1} \text{POSTDIV2}}. \quad (6)$$

La documentación del RP2040 establece las siguientes restricciones del PLL [?]:

$$f_{\text{ref}} \geq 5 \text{ MHz}, \quad (7)$$

$$750 \text{ MHz} \leq f_{\text{VCO}} \leq 1.600 \text{ MHz}, \quad (8)$$

$$16 \leq \text{FBDIV} \leq 320, \quad (9)$$

$$1 \leq \text{POSTDIV1}, \text{POSTDIV2} \leq 7. \quad (10)$$

Se adopta además $POSTDIV1 \geq POSTDIV2$, tal como recomienda la documentación.

Con $f_{XOSC} = 12 \text{ MHz}$, la primera restricción implica

$$\frac{12}{REFDIV} \geq 5 \implies \boxed{REFDIV \in \{1, 2\}}. \quad (11)$$

Por consiguiente, fijar $REFDIV=1$ no era una condición necesaria: era solamente una reducción del espacio de búsqueda. Permitir $REFDIV=2$ duplica la resolución disponible en la síntesis del VCO y genera soluciones adicionales válidas.

El intervalo 125 MHz a 250 MHz para f_{SYS} es un requisito experimental del proyecto PixiePico. No debe confundirse con la frecuencia nominal garantizada por la especificación del RP2040: la zona superior del intervalo constituye overclocking y debe validarse sobre el hardware concreto.

3. Función objetivo conjunta

Combinando (??) y (??) se obtiene

$$\boxed{f_{\text{out}} = \frac{128 f_{XOSC} FBDIV}{REFDIV POSTDIV1 POSTDIV2 N}} \quad (12)$$

Para una frecuencia objetivo f_T , el problema consiste en encontrar la tupla entera

$$(REFDIV, FBDIV, POSTDIV1, POSTDIV2, N) \quad (13)$$

que minimiza

$$E = |f_{\text{out}} - f_T|, \quad (14)$$

sujeta a todas las restricciones anteriores y a

$$125 \text{ MHz} \leq f_{SYS} \leq 250 \text{ MHz}. \quad (15)$$

No es apropiado optimizar primero el PLL y luego la PIO de manera independiente. Una frecuencia f_{SYS} que no sea la más cercana a un valor elegido a priori puede producir un error final menor al combinarse con cierto valor entero N .

4. Algoritmo de búsqueda exhaustiva

El espacio de parámetros es discreto y pequeño. Por ello, una enumeración exhaustiva entrega el óptimo global dentro del modelo, sin depender de semillas, gradientes ni mínimos locales.

Para cada cuádrupla válida de parámetros PLL se calcula

$$N^* = \frac{128 f_{SYS}}{f_T}. \quad (16)$$

Como el error es monótono a cada lado de N^* , basta evaluar $\lfloor N^* \rfloor$ y $\lceil N^* \rceil$. No es necesario recorrer los más de dieciséis millones de valores posibles del registro PIO.

Se recomienda usar aritmética racional o enteros suficientemente anchos. El uso prematuro de `float` puede cambiar el orden de candidatos con errores muy próximos.

Listing 1: Implementación de referencia del optimizador

```
from fractions import Fraction

XOSC = 12_000_000
SYS_MIN = 125_000_000
SYS_MAX = 250_000_000
VCO_MIN = 750_000_000
```

```

VCO_MAX = 1_600_000_000

def optimize(target_hz):
    best = None

    for refdiv in range(1, 64):
        ref = Fraction(XOSC, refdiv)
        if ref < 5_000_000:
            continue

        for fbdiv in range(16, 321):
            vco = ref * fbdiv
            if not (VCO_MIN <= vco <= VCO_MAX):
                continue
            if ref > vco / 16:
                continue

            for postdiv1 in range(1, 8):
                for postdiv2 in range(1, postdiv1 + 1):
                    sys = vco / (postdiv1 * postdiv2)
                    if not (SYS_MIN <= sys <= SYS_MAX):
                        continue

                    ideal_n = 128 * sys / target_hz
                    n0 = ideal_n.numerator // ideal_n.denominator

                    for n in {n0, n0 + 1}:
                        if not (256 <= n <= 0xFFFFF):
                            continue

                        actual = 128 * sys / n
                        signed_error = actual - target_hz

                        # Mayor VCO como desempate: menor jitter.
                        key = (abs(signed_error), -vco)
                        candidate = (key, refdiv, fbdiv,
                                    postdiv1, postdiv2,
                                    n, vco, sys,
                                    actual, signed_error)

                        if best is None or key < best[0]:
                            best = candidate

    return best

```

5. Resultados

Cuadro 1: Configuraciones óptimas encontradas mediante búsqueda exhaustiva.

f_T (Hz)	REF	FB	PD1	PD2	N	f_{SYS} (Hz)	Error (Hz)
7.074.000	2	263	7	1	4079	225.428.571,429	+2,731762
14.074.000	2	211	3	2	1919	211.000.000,000	-3,126628
28.074.000	2	261	4	3	595	130.500.000,000	-50,420168

5.1. Caso $f_T = 7,074 \text{ MHz}$

La mejor tupla es

$$(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 263, 7, 1, 4079). \quad (17)$$

De ella se obtiene

$$f_{\text{ref}} = \frac{12 \text{ MHz}}{2} = 6 \text{ MHz}, \quad (18)$$

$$f_{\text{VCO}} = 6 \text{ MHz} \times 263 = 1.578 \text{ MHz}, \quad (19)$$

$$f_{\text{SYS}} = \frac{1.578 \text{ MHz}}{7} = 225,428.571.428.571 \text{ MHz}. \quad (20)$$

El divisor PIO es

$$D = \frac{4079}{256} = 15 + \frac{239}{256} = 15,93359375. \quad (21)$$

Finalmente,

$$f_{\text{out}} = 7.074.002,731.762 \text{ Hz}, \quad (22)$$

$$\Delta f = \boxed{+2,731.762 \text{ Hz}}, \quad (23)$$

$$\varepsilon = \boxed{+0,386169 \text{ ppm}}. \quad (24)$$

Como comprobación, 233 MHz también resulta sintetizable cuando se permite **REFDIV=2**: (2, 233, 3, 2) produce un VCO de 1.398 MHz y $f_{\text{SYS}} = 233 \text{ MHz}$. Con $N = 4216$, su error es +3,795.066 Hz, ligeramente mayor que el óptimo global.

5.2. Caso $f_T = 14,074 \text{ MHz}$

La mejor tupla es

$$(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 211, 3, 2, 1919). \quad (25)$$

Los relojes y el divisor PIO son

$$f_{\text{ref}} = 6 \text{ MHz}, \quad (26)$$

$$f_{\text{VCO}} = 1.266 \text{ MHz}, \quad (27)$$

$$f_{\text{SYS}} = \frac{1.266 \text{ MHz}}{6} = 211 \text{ MHz}, \quad (28)$$

$$D = \frac{1919}{256} = 7 + \frac{127}{256} = 7,49609375. \quad (29)$$

La salida resultante es

$$f_{\text{out}} = \frac{128(211000000)}{1919} = 14.073.996,873.372 \text{ Hz}, \quad (30)$$

$$\Delta f = \boxed{-3,126.628 \text{ Hz}}, \quad (31)$$

$$\varepsilon = \boxed{-0,222156 \text{ ppm}}. \quad (32)$$

5.3. Caso $f_T = 28,074 \text{ MHz}$

Una de las tuplas óptimas es

$$(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 261, 4, 3, 595). \quad (33)$$

Los valores asociados son

$$f_{\text{ref}} = 6 \text{ MHz}, \quad (34)$$

$$f_{\text{VCO}} = 1.566 \text{ MHz}, \quad (35)$$

$$f_{\text{SYS}} = \frac{1.566 \text{ MHz}}{12} = 130,5 \text{ MHz}, \quad (36)$$

$$D = \frac{595}{256} = 2 + \frac{83}{256} = 2,32421875. \quad (37)$$

La frecuencia obtenida es

$$f_{\text{out}} = \frac{128(130500000)}{595} = 28.073.949,579.832 \text{ Hz}, \quad (38)$$

$$\Delta f = \boxed{-50,420.168 \text{ Hz}}, \quad (39)$$

$$\varepsilon = \boxed{-1,795974 \text{ ppm}}. \quad (40)$$

Existen varias tuplas con el mismo error para este objetivo. Por ejemplo, el mismo VCO de 1.566 MHz con $(POSTDIV1, POSTDIV2, N) = (6, 2, 595)$, $(5, 2, 714)$ o $(7, 1, 1020)$ conserva la misma razón final. La selección entre ellas puede realizarse según frecuencia del sistema, consumo y compatibilidad con el resto del firmware.

6. Interpretación y limitaciones

1. La búsqueda obtiene el óptimo global *estático* dentro de las restricciones declaradas. No garantiza error cero porque todos los divisores relevantes son enteros o racionales con granularidad finita.
2. El valor nominal calculado no incluye tolerancia, deriva térmica ni envejecimiento del cristal de 12 MHz. En la práctica, varios ppm del cristal pueden superar el error matemático residual del algoritmo.
3. Una frecuencia racional como 1.578 MHz/7 es físicamente válida. Sin embargo, las API del SDK que almacenan frecuencias en hertz enteros pueden reportarla redondeada. La configuración del PLL debe realizarse a partir de sus parámetros exactos, no reconstruirse desde el entero redondeado.
4. Cambiar PLL_SYS durante la ejecución afecta CPU, PIO y los periféricos derivados de ese reloj. Deben revisarse temporizadores, baud rates, flash y cualquier lógica que suponga una frecuencia fija. El PLL USB es independiente, aunque el firmware USB puede verse afectado indirectamente si el cambio no se secuencia correctamente.
5. Para errores menores que el óptimo estático puede alternarse entre dos divisores PIO adyacentes en una proporción calculada. Esto reduce el error promedio, pero introduce modulación determinista y componentes laterales; por ello constituye otro problema de diseño y no debe confundirse con una frecuencia estática exacta.

7. Recomendación de implementación

El firmware debería conservar los cinco parámetros del candidato ganador, no solo f_{SYS} y D . La estructura de resultado debe incluir como mínimo `refdiv`, `fbdiv`, `postdiv1`, `postdiv2`, `pio_div_int`, `pio_div_frac`, frecuencia calculada y error. Antes de aplicar el resultado, debe verificarse nuevamente cada restricción y configurar el PLL mediante sus parámetros exactos.

Para el primer ensayo de hardware se recomienda comenzar con 14,074 MHz, porque su solución usa un f_{SYS} entero de 211 MHz y ofrece el menor error relativo de los tres casos. Después

debe medirse la salida con un contador referenciado; sin esa medición, optimizar por debajo de un ppm es precisión matemática, no necesariamente precisión física.

Referencias

- [1] Raspberry Pi Ltd., *RP2040 Datasheet: A microcontroller by Raspberry Pi*, sección 2.18, “PLL”, versión de febrero de 2025. Disponible en: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>.
- [2] Raspberry Pi Ltd., *Pico SDK—hardware_pll*, implementación de `pll_init`. Disponible en: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_pll/pll.c.